

# Lab 5 — Choropleth + Bivariate + Cartogram Mapping (GeoPandas + matplotlib)

*Open-Source Geovisualization • GEOG 8005 / ESCI 6000 • Feb 16, 2026*

## Overview

In this lab you will build three classic thematic map types in Python: (1) a univariate choropleth, (2) a bivariate choropleth that communicates two variables at once, and (3) a cartogram that distorts polygon areas by an attribute. The emphasis is code literacy: you will run provided demo code, explain what each block does, and identify which parameters control key visual design choices (e.g., color, classification, line width, legends, and distortion strength).

## What you will produce (portfolio-friendly)

- A clean univariate choropleth map (PNG) with a well-labeled legend
- A bivariate choropleth map (PNG) plus a 2D legend/key
- One cartogram map (PNG) (contiguous cartogram recommended; non-contiguous fallback allowed)
- A short set of written answers to the code-reading questions (can be included as screenshots)

## Prerequisites

- A working conda environment (e.g., geoviz) with: geopandas, pandas, numpy, matplotlib, mapclassify
- Optional (recommended): contextily (basemaps) and cartogram (contiguous cartograms)
- A polygon layer you can map (GeoPackage/GeoJSON/Shapefile) that contains at least TWO numeric attributes (or a CSV you can join to add them)

```
# In terminal (example)
conda activate geoviz
conda install -c conda-forge geopandas pandas numpy matplotlib mapclassify
conda install -c conda-forge contextily # optional basemaps
pip install cartogram # optional cartograms
python -c "import geopandas, mapclassify; print('OK')"
```

## Data for this lab

Use ONE of the following options:

### Option A (recommended): Use your course project polygons

- Polygons: your study areas, tracts, counties, districts, etc. (GeoJSON/GPKG/Shapefile)
- Attributes: two numeric fields to map (e.g., population + income; incidence + accessibility) OR a CSV with those fields to join by an ID

### Option B: Use a small sample dataset (if you do not have project data yet)

- Install geodatasets (optional): pip install geodatasets
- Then load a sample polygon dataset and choose two numeric fields from its attribute table

```
# Example: load a sample dataset (requires: pip install geodatasets)
import geopandas as gpd
import geodatasets

gdf = gpd.read_file(geodatasets.get_path("geoda.chicago_commpop")) # example
print(gdf.columns)
print(gdf.head())
```

## Submission (screenshots only)

Submit screenshots:

1. Successful imports + package versions (GeoPandas, matplotlib, mapclassify; cartogram if used)
2. Data inspection output: CRS + columns + head() + basic missing-value check
3. Univariate choropleth map + notes on the scheme (e.g., Quantiles, k=5) and the field mapped
4. Bivariate choropleth map + bivariate legend/key
5. Cartogram map + the field used to distort area (and average\_error if using cartogram)
6. Your answers to the code-reading questions (can be screenshots of a markdown file, notebook, or terminal)

## Part 1 — Setup + data loading

### 1.1 Import + environment check

- Import core packages and print versions (this helps debugging and reproducibility).
- If an import fails, fix your environment before continuing.

```
import geopandas as gpd
import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt

import mapclassify # required for scheme=... in GeoPandas plots

print("geopandas:", gpd.__version__)
print("pandas:", pd.__version__)
print("numpy:", np.__version__)
print("matplotlib:", matplotlib.__version__)
print("mapclassify:", mapclassify.__version__)
```

## 1.2 Read + inspect polygons

- Load your polygon layer and confirm the CRS and attribute fields are what you expect.
- Confirm your two numeric mapping fields are present (or plan to join them).

```
import geopandas as gpd

gdf = gpd.read_file("data/raw/polygons.gpkg") # <-- change path
print("CRS:", gdf.crs)
print("Bounds:", gdf.total_bounds)
print("Columns:", list(gdf.columns))
print(gdf.head())
```

## 1.3 Choose a working projected CRS

- For choropleths and cartograms, prefer a projected CRS (meters) and (ideally) an equal-area projection.
- Reproject once at the beginning and keep your working GeoDataFrame consistent.

```
# Example equal-area CRS (USA): NAD83 / Conus Albers (EPSG:5070)
# If you are outside the USA, choose an equal-area projection appropriate for your
region.
WORK_EPSG = "EPSG:5070" # example only

gdf_m = gdf.to_crs(WORK_EPSG)
```

### Check your understanding (Part 1)

7. What does CRS mean, and why does it matter for mapping and area-based distortion?
8. What does `gdf.total_bounds` return, and how would you use it to sanity-check your data?
9. Which line reprojects the data, and what would happen if `WORK_EPSG` is invalid?

## Part 2 — Univariate choropleth

### 2.1 Clean the field you will map

- Choose ONE numeric field (e.g., population, rate, density).
- Convert to numeric and handle missing values explicitly.

```
VAR = "your_numeric_field" # <-- change
gdf_m[VAR] = pd.to_numeric(gdf_m[VAR], errors="coerce")

print("Missing values:", gdf_m[VAR].isna().sum())
print(gdf_m[VAR].describe())
```

### 2.2 Plot a choropleth with a classification scheme

- Use `scheme + k` to classify values into bins (requires `mapclassify`).

- Make design choices explicit: color map, boundaries, transparency, title.

```
import matplotlib.pyplot as plt

ax = gdf_m.plot(
    column=VAR,
    scheme="Quantiles", # try: "EqualInterval", "FisherJenks"
    k=5,
    cmap="Blues",
    linewidth=0.3,
    edgecolor="white",
    legend=True,
    figsize=(9, 7),
)
ax.set_title(f"Choropleth: {VAR} (Quantiles, k=5)")
ax.set_axis_off()
plt.show()
```

## 2.3 Optional: add a basemap (contextily)

- Basemaps require Web Mercator (EPSG:3857). Reproject a COPY to 3857 just for plotting.
- Basemap tiles require an internet connection.

```
# Optional basemap (requires: contextily)
import contextily as cx

gdf_web = gdf_m.to_crs(epsg=3857)

ax = gdf_web.plot(
    column=VAR,
    scheme="Quantiles",
    k=5,
    cmap="Blues",
    linewidth=0.3,
    edgecolor="white",
    legend=True,
    figsize=(9, 7),
)
cx.add_basemap(ax, source=cx.providers.CartoDB.Positron)
ax.set_title(f"Choropleth + basemap: {VAR}")
ax.set_axis_off()
plt.show()
```

## Check your understanding (Part 2)

10. Which parameter controls the number of classes (bins)?
11. Which parameter changes the color palette? Give two valid cmap examples.
12. Which parameters control polygon boundary appearance (color and line width)?
13. What does scheme='Quantiles' mean, and how is it different from 'EqualInterval'?

## Part 3 — Bivariate choropleth

### 3.1 Create bivariate classes (3x3 quantiles)

- Pick TWO numeric fields: X and Y.
- Bin each field into 3 quantile classes, then combine into a single bivariate class.

```
X = "your_field_x" # <-- change
Y = "your_field_y" # <-- change
N = 3 # number of bins per variable (3 -> 3x3)

gdf_m[X] = pd.to_numeric(gdf_m[X], errors="coerce")
gdf_m[Y] = pd.to_numeric(gdf_m[Y], errors="coerce")

# Quantile bins (0..N-1). duplicates='drop' avoids errors if many ties.
gdf_m["x_bin"] = pd.qcut(gdf_m[X], N, labels=False, duplicates="drop")
gdf_m["y_bin"] = pd.qcut(gdf_m[Y], N, labels=False, duplicates="drop")

# Combine bins into one class ID (row-major)
gdf_m["bi_class"] = gdf_m["y_bin"] * N + gdf_m["x_bin"]
```

### 3.2 Map the bivariate classes to colors

- Use a 3x3 color matrix (9 colors).
- You will be asked to identify which line(s) to change to adjust the palette.

```
# 3x3 bivariate palette (row-major: y from low->high, x from low->high)
palette = [
    "#e8e8e8", "#ace4e4", "#5ac8c8",
    "#dfb0d6", "#a5add3", "#5698b9",
    "#be64ac", "#8c62aa", "#3b4994",
]

def class_to_color(v):
    if pd.isna(v):
        return "#dddddd"
    return palette[int(v)]

gdf_m["bi_color"] = gdf_m["bi_class"].apply(class_to_color)

ax = gdf_m.plot(color=gdf_m["bi_color"], linewidth=0.3, edgecolor="white",
figsize=(9, 7))
ax.set_title(f"Bivariate choropleth: {X} (x) vs {Y} (y)")
ax.set_axis_off()
plt.show()
```

### 3.3 Build a bivariate legend (2D key)

- A bivariate legend is a small 2D grid that explains how colors map to low/medium/high for X and Y.
- You can screenshot the legend alone or place it next to your map in a report/notebook.

```

from matplotlib.patches import Rectangle

fig, ax = plt.subplots(figsize=(4, 4))

for y in range(N):
    for x in range(N):
        idx = y * N + x
        ax.add_patch(Rectangle((x, y), 1, 1, facecolor=palette[idx],
                                edgecolor="none"))

ax.set_xlim(0, N)
ax.set_ylim(0, N)
ax.set_aspect("equal")

# Minimal labeling (edit these to match your variables)
ax.set_xticks([0.5, 1.5, 2.5])
ax.set_yticks([0.5, 1.5, 2.5])
ax.set_xticklabels(["Low", "Med", "High"])
ax.set_yticklabels(["Low", "Med", "High"])
ax.set_xlabel(X + " →")
ax.set_ylabel(Y + " ↑")

# Clean look
for spine in ax.spines.values():
    spine.set_visible(False)

plt.title("Bivariate legend (3x3)")
plt.show()

```

### Check your understanding (Part 3)

14. What does `pd.qcut` do, and why might `duplicates='drop'` be necessary?
15. How is `bi_class` computed? Explain the formula in one sentence.
16. Which lines would you change to switch from a 3x3 to a 4x4 bivariate map?
17. Which variable controls the colors used for the map?

## Part 4 — Cartograms

### 4.1 Prepare the cartogram field

- Choose ONE positive numeric field (absolute counts work best: population, cases, trips).
- Cartograms distort area, so use a projected equal-area CRS.

```

CART_FIELD = "your_cartogram_field" # <-- change

gdf_m[CART_FIELD] = pd.to_numeric(gdf_m[CART_FIELD], errors="coerce")
gdf_m = gdf_m.dropna(subset=[CART_FIELD]).copy()

# Cartograms require positive values
gdf_m = gdf_m[gdf_m[CART_FIELD] > 0].copy()

```

```
print(gdf_m[CART_FIELD].describe())
```

## 4.2 Contiguous cartogram (recommended: cartogram package)

- This uses an iterative continuous cartogram approach (python package: cartogram).
- Tune distortion quality vs runtime using max\_iterations and max\_average\_error.

```
# Requires: pip install cartogram
import cartogram
import matplotlib.pyplot as plt

c = cartogram.Cartogram(
    gdf_m,
    CART_FIELD,
    max_iterations=60,      # try: 30, 60, 99
    max_average_error=0.03 # try: 0.10 (faster) -> 0.02 (more accurate)
)

print("Average residual area error:", c.average_error)

ax = c.plot(
    column=CART_FIELD,
    scheme="Quantiles",
    k=5,
    cmap="Reds",
    linewidth=0.3,
    edgecolor="white",
    legend=True,
    figsize=(9, 7),
)
ax.set_title(f"Contiguous cartogram by {CART_FIELD}")
ax.set_axis_off()
plt.show()

# Optional export
c.to_file("outputs/cartogram.gpkg", layer="cartogram", driver="GPKG")
```

## 4.3 Fallback: simple non-contiguous cartogram (scale polygons)

- If the cartogram package fails to install, you can still create a non-contiguous cartogram by scaling each polygon.
- This preserves polygon shape but breaks adjacency (polygons may overlap or separate).

```
from shapely.affinity import scale

vals = gdf_m[CART_FIELD].to_numpy()
factors = np.sqrt(vals / np.nanmean(vals)) # area is ~factor^2

scaled_geoms = [
    scale(geom, xfact=f, yfact=f, origin="center")
```

```

    for geom, f in zip(gdf_m.geometry, factors)
]

gdf_scaled = gdf_m.copy()
gdf_scaled["geometry_scaled"] = scaled_geoms
gdf_scaled = gdf_scaled.set_geometry("geometry_scaled")

ax = gdf_scaled.plot(
    column=CART_FIELD,
    scheme="Quantiles",
    k=5,
    cmap="Reds",
    linewidth=0.3,
    edgecolor="white",
    legend=True,
    figsize=(9, 7),
)
ax.set_title(f"Non-contiguous cartogram (scaled) by {CART_FIELD}")
ax.set_axis_off()
plt.show()

```

### Check your understanding (Part 4)

18. Why must the cartogram field be positive?
19. What do `max_iterations` and `max_average_error` control?
20. What is `c.average_error` measuring (in plain language)?
21. In the fallback method, why do we use `sqrt(value/mean)` rather than `value/mean` directly?

### Checklist before you submit

- You can run every code block without errors (or you clearly document any optional steps you skipped).
- Your maps have clear titles and the legends match the field(s) mapped.
- You can point to the exact parameter(s) that control: (a) color palette, (b) number of classes, (c) polygon boundary styling.
- Your bivariate legend matches your bivariate palette and binning choice.
- If you used the cartogram package, you recorded `average_error` and your chosen parameters.